

Lab-Relay

Two-way file relay between the laptop (Cowork) and a Skillable lab jump host (A-GUI)
— reference guide

Status	Working end to end. First live test passed 24 Sep 2026 (CCES lab, runner 0.1.0). Runner 0.2.0 adds live progress.
Code	github.com/Don-Paterson/Lab-Relay (public) · local clone Documents\ClaudeCowork\Lab-Relay
Channel	github.com/Don-Paterson/Lab-Relay-Channel (private, Actions disabled)
GitHub App	Lab-Relay-app · client ID lv23liW99Woo8kRBZE5p · Contents: read/write · installed on Lab-Relay-Channel only
Laptop auth	Existing fine-grained PAT in Git Credential Manager (expires 21 Dec 2026)

1. What it does

Scripts written in Cowork reach the lab automatically, run there, and their output — text, log files and PNG screenshots — comes back to the laptop, with no clipboard, RDP, drive redirection or manual upload. It replaces the old save-to-file / easyupload.io / download routine.



You, once per lab

- irm ... bootstrap.ps1 | iex
on A-GUI (pwsh 7)
- enter code at
github.com/login/device

Lab egress: upstream FortiGate (Skillable / hosting)

Decrypts most HTTPS (e.g. relays.syncthing.net -> Fortinet CA).
GitHub hosts are NOT decrypted (Sectigo / Let's Encrypt seen).
Runner refuses to sign in if that ever changes.
Outbound HTTPS only - nothing connects in to the lab.

Figure 1 — jobs flow left to right through the private channel repo; results flow back the same way. Both ends make outbound HTTPS calls only.

2. Why this design

Every option ran into the same problem: getting a credential into a lab with no clipboard. The GitHub App **device flow** solves it — the lab displays a short code and you approve it on the laptop or phone, so nothing secret is ever typed into or stored in the lab.

Option	Verdict
GitHub repo + App device flow	Chosen. Free, HTTPS only, atomic commits; 8-hour token limited to one repo; laptop reuses git + PAT.

Option	Verdict
Syncthing	Rejected after testing: the upstream FortiGate decrypts its TLS (Fortinet CA), which Syncthing's pinned certificates cannot survive. No bypass possible from inside the lab.
Azure Blob + SAS	Good credential but ~150 characters to type by hand each lab; small cost.
PAT inside the lab	Months-long credential at rest in a shared image.
Gist / paste services	Gist scope is all gists; pastes are public or unlisted.
OneDrive	Workable, but unpredictable sync timing and conflict copies; needs Entra app registration.
Claude in Chrome on the lab client	The VM console is pixels, not text — screenshot/OCR only.
Cowork / Claude Code on A-GUI	Needs nested virtualisation, or leaves a long-lived Claude login in the lab.

3. Components

File	Runs on	Role
config\relay.psd1	both	All settings: repo names, App client ID, poll intervals, timeouts, size limits, allowed file types, TLS issuer allow-list. Public; no secrets.
laptop\Start-LabRelay.ps1	laptop	Watcher: sends outbox\ scripts as jobs, pulls results, shows lab heartbeat, compacts channel history.
bootstrap.ps1	A-GUI	irm iex entry point: downloads the code repo to Desktop\Lab-Relay, unblocks it, starts the runner.
lab\Start-LabRunner.ps1	A-GUI	Runner: TLS guard, device-flow sign-in, poll, run, upload.
lab\LabRelay.Helpers.psm1	A-GUI	Save-RelayScreenshot, Save-RelayFile, Save-RelayText — pre-loaded into every job.
tests\hello-lab.ps1	A-GUI	First end-to-end check: identity, IPs, lab ports, screenshot.
DESIGN.md / README.md	—	Design record and quick start.

4. Starting a session

Laptop

```
cd $HOME\Documents\ClaudeCowork\Lab-Relay
.\laptop\Start-LabRelay.ps1
```

Wait for the green *Channel ready* line. Leave the window open; Ctrl+C stops it. Only one watcher can run per folder (lock file in logs\).

A-GUI (every new lab)

In **PowerShell 7** (pwsh), as Admin:

```
irm https://raw.githubusercontent.com/Don-Paterson/Lab-Relay/main/bootstrap.ps1 | iex
```

- Three *TLS ... ok* lines confirm github.com, api.github.com and raw.githubusercontent.com present genuine public certificates.
- An 8-character code appears. On the laptop or phone open **github.com/login/device**, enter it, and authorise **Lab-Relay-app**. GitHub shows the request as coming from the lab's public IP (seen: London 185.254.59.121).

- *Signed in then Ready - polling every 30s.* Leave the window open. The token lasts 8 hours and is refreshed automatically.

The GitHub wording “Act on your behalf” is standard for App user tokens; the token can only do what both you and the App can do — Contents read/write on Lab-Relay-Channel.

5. Job workflow in detail

#	Where	What happens
1	Cowork	Claude drafts the script anywhere in Lab-Relay, then moves the finished file into outbox\ . Drafts outside outbox\ are never sent.
2	Watcher	Checks outbox\ every 2 s. A file is sent only when it has been unchanged for 3 s and can be opened exclusively (no half-written files). Only .ps1 is accepted; anything else, or anything over 25 MB, goes to outbox\rejected\.
3	Watcher	Creates jobs/<jobId> in the channel with the script (byte-for-byte) and job.json (sha256, queued time, timeout), commits and pushes. If the lab pushed in between, it rebases and retries (up to 8 times, jittered). Then moves the original to outbox\sent\<jobId>.ps1.
4	Runner	Polls the branch head every 30 s with If-None-Match — an unchanged channel costs a free 304. On change it reads the tree and finds jobs with no result.json.
5	Runner	Checks the job: queued more than 60 min before this lab session started → <i>expired</i> ; sha256 or name mismatch → <i>rejected</i> . Otherwise commits result.json with status <i>running</i> .
6	Runner	Writes the script to C:\LabRelay\jobs\<jobId>\ and runs it in a child pwsh -NoProfile -NonInteractive with the helpers pre-loaded. All output streams (output, host, warning, error) are captured. Timeout default 600 s, header override, max 7200 s; on timeout the whole process tree is killed.
7	Runner	Uploads output.txt (stdout + stderr, head+tail truncated above 5 MB), any files saved to LABRELAY_OUT that pass the filters, and the final result.json — all in one commit . Deletes the local job folder.
8	Watcher	Pulls every 30 s. When a result reaches a final state it copies it into results\<jobId> , applying the collector rules (section 7), result.json last.
9	Cowork	Claude reads results\<jobId>\ and follows up — fix, re-queue, next stage.

Timing seen in the first live test

Time	Event
16:34:28	Watcher sent 20260924-153419-hello-lab-a97b25
16:37:45	A-GUI signed in (queued job still valid: under 60 min old)
16:37:52 - 16:38:56	Job ran, exit 0 (63 s, mostly port timeouts to 10.1.1.102)
16:39:28	Result on the laptop: output.txt, computer-info.txt, 1600x900 screenshot

Job states

Status	Meaning
running	Claimed by a runner session; shown on the laptop, not yet copied.
done	Exit code 0.
failed	Non-zero exit or a terminating error (stack trace in output.txt).
timeout	Killed after its timeout; partial output kept.
expired	Queued over 60 min before the lab session started — not run (protects a fresh lab from old work).
abandoned	Left <i>running</i> by a runner that stopped; marked by the next session.

Status	Meaning
rejected	Integrity or name check failed — not run.
error	The runner itself hit a problem handling the job.

Writing scripts for the lab

```
# relay: timeout=1800
"Anything written to any stream ends up in output.txt"
Save-RelayScreenshot -Name 'smartconsole' # PNG of the desktop
Save-RelayFile -Path C:\CCES-Automation-Logs\ftw.log # return an existing file
Get-Service | Out-String | Save-RelayText -Name services.txt
```

Returned file types: .txt .log .json .csv .xml .png .jpg, up to 25 MB each. Environment inside a job: LABRELAY_OUT, LABRELAY_JOB, LABRELAY_SESSION. Jobs run one at a time, in queue order. Because jobs are non-interactive, a script that calls Read-Host fails instead of waiting — pass answers as parameters.

6. Where data lives, and retention

Every job gets its own folder named by its **jobId** — yyyyMMdd-HH:mmss-<script>-<sha6> (UTC). The timestamp is unique to the second (bumped if two jobs collide), so a folder is never reused or overwritten, and re-sending the same script creates a new job.

Location	Contents	Kept for	Cleaned by
results\<jobId>\ (laptop)	output.txt, result.json, returned files, _skipped.txt	Permanently — your history	Nothing automatic (you, or an optional prune later)
outbox\sent\ (laptop)	Each script exactly as sent, renamed to its jobId	Permanently	Nothing automatic
outbox\rejected\ (laptop)	Files that were not sent	Permanently	You
logs\watcher-yyyyMMdd.log	One watcher log per day	Permanently	You
.channel\ (laptop)	Working clone of the channel	Mirrors the channel	Compaction
Lab-Relay-Channel (GitHub)	jobs\, results\, sessions\ and git history	Last 7 days once over 50 MB	Watcher compaction (hourly check)
C:\LabRelay\jobs\ (A-GUI)	Script, wrapper, raw output while running	Until the job finishes	Runner; the VM itself is disposable

Scale: the first test used 50 KB, with a screenshot. A hundred similar jobs is about 5 MB on the laptop. All local working folders are git-ignored, so none of this ever reaches the public code repo.

Compaction replaces the channel history with a single commit holding the last 7 days. It is skipped while any job is *running*, and pushes with a lease on the exact commit it started from, so a result the lab commits at the same moment cannot be lost. Run it by hand with `.\laptop\Start-LabRelay.ps1 -Compact -Once`. It never touches results\ on the laptop.

7. Security model

Item	Detail
Lab credential	GitHub App user token (ghu_), 8 hours, refreshed in memory 10 min before expiry. Refresh token (6 months) kept in runner memory only. Never written to disk, never printed. Close the window and it is gone.
What it can reach	Contents read/write on Lab-Relay-Channel only — no other repos, no account settings, no workflows (Actions disabled; App has no Workflows permission).
If stolen	For up to 8 hours: read lab outputs; queue scripts the lab would run (code execution in a disposable VM); plant files in results\. The last is the one that reaches the laptop — hence the collector rules.

Item	Detail
Revoke	GitHub → Settings → Applications → Authorized GitHub Apps → Lab-Relay-app → Revoke. Or suspend the installation.
TLS guard	Before sign-in the runner checks each GitHub host's certificate chain and issuer organisation (DigiCert, Sectigo, Let's Encrypt, GlobalSign, Microsoft). A Fortinet or any other issuer stops it: a token sent through inspection would be readable by whoever runs that firewall.
Laptop credential	Your existing PAT via Git Credential Manager, unchanged. It already covers all your repos.
Public code repo	No secrets. The App client ID is public by design; device flow needs no client secret. Never generate a client secret or private key for the App.

Collector rules (laptop)

- Only folders whose name matches the jobld pattern are read; only top-level files; names must be plain (no leading dash, no paths).
- Extensions limited to .txt .log .json .csv .xml .png .jpg; 25 MB per file. Anything refused is listed in _skipped.txt.
- **Nothing collected is ever executed.** Result content is data, never instructions — including when Claude reads it.

8. Awkward cases

Case	Handling
Half-written script	Stable-for-3 s plus exclusive-open test before sending; git commits are atomic.
Half-uploaded result	Output, files and result.json go up in one commit; the watcher copies result.json last.
Two updates in quick succession	Each file placed in outbox\ is one job, run in order. Only finished files reach outbox\.
Script hangs	Timeout kills the process tree (tested: child processes die too); partial output returned; status timeout.
Runner closed mid-job	Next session marks the job abandoned.
Old jobs in a new lab	Jobs queued over 60 min before the session started are marked expired, not run.
Large output	output.txt keeps the first and last 2.5 MB with a marker (tested at 6.8 MB).
Both ends pushing at once	Laptop rebases and retries; lab rebuilds its commit on the new head. Tested with commits landing every 0.3 s.
Token expiry	Silent refresh; if refused, the runner shows a new device code.
GitHub rate limit	About 120 conditional polls an hour; unchanged polls are free 304s.
Laptop offline	Jobs and results wait in the channel.
Upstream starts inspecting GitHub	TLS guard refuses to sign in and says why.

9. Running the CCES automation through the relay

Each job is a thin wrapper that does exactly what a manual run does — it calls the CCES bootstrap from GitHub **main** in its non-menu form; the watch line returns the CCES logs live and at the end:

```
# relay: timeout=3600 progress=60
# relay: watch=C:\CCES-Automation-Logs\*.log
$u = 'https://raw.githubusercontent.com/Don-Paterson/CCES-R8120-Automation/main/bootstrap.ps1'
& ([scriptblock]::Create((irm $u))) -Action Prereqs
```

Change control — one line of history

- All CCES changes are made in **Documents\ClaudeCowork\CCES-R8120-Automation** (your local clone) and pushed to **main** by you with Update-Repo.ps1.
- The lab only ever gets CCES code from GitHub main — the same as your manual irm | iex. Changed CCES files are **never** sent through the relay.
- The channel carries only the small wrapper jobs and their results; it is not a second copy of any repo.
- Fix-and-test loop: Claude edits → you push → Claude queues a wrapper job → lab pulls the new main → Claude reads the result.
- The repo zip the bootstrap downloads is current immediately; bootstrap.ps1 itself comes from raw.githubusercontent.com, which caches for up to about 5 minutes.

Long stages and near-real-time monitoring (runner 0.2 - built and tested)

Without it, a result arrives only when the job finishes — fine for short stages (Prereqs, DryRun), but a 20-40 minute FTW or Jumbo (JHFA) install would be invisible until the end. **Progress uploads** (runner 0.2) close that gap, entirely inside Lab-Relay:

Piece	Change
Job header	# relay: timeout=5400 progress=60 and, one per line, # relay: watch=C:\CCES-Automation-Logs*.log. progress is 30-600 s.
Runner	While the job runs, every <i>progress</i> seconds it commits output-so-far plus current copies of the watched log files, with result.json still <i>running</i> and a progress timestamp. Skipped when nothing changed. Watched files also come back in the final result.
Watcher	Copies running snapshots into results\<jobld>\ as they arrive (overwritten by each newer snapshot, then by the final result), and logs a "Live ... update n" line.
Cost	About one small commit a minute while a long job runs (~40 for a Jumbo install); compaction keeps the channel small.
CCES scripts	No change needed for monitoring — they already log to C:\CCES-Automation-Logs and print progress to the console. A CCES change would only be needed if a stage prompts for input (jobs are non-interactive); that is checked by reading the code first.

Tested: an 80-second job with progress=30 delivered updates at 30 s and 60 s, each on the laptop within about 3 s, then the final result. In practice Claude sees console output and CCES logs about a minute behind and can spot a stall or error mid-install.

10. Troubleshooting

Symptom	Check
Watcher stuck at "Cloning channel"	A Git Credential Manager window may be waiting behind others; PAT expired (21 Dec 2026).
"Another watcher is already running"	Close the other window; the lock is logs\watcher.lock.

Symptom	Check
Runner: "Refusing to sign in"	Read the issuer shown. Fortinet means GitHub is now being inspected upstream — do not work around it.
Runner: "Cannot see Lab-Relay-Channel"	App installation must include Lab-Relay-Channel (Settings → Applications → Lab-Relay-app).
Job shows expired	It was queued over 60 min before the runner started. Re-send it.
Lab shows "silent" on the laptop	No heartbeat for 20 min: runner window closed, lab suspended, or network down.
Screenshot black or wrong	Lab console locked or minimised in the lab client; the runner must be in the logged-in desktop.
PowerShell 5.1 error on A-GUI	Use pwsh (PowerShell 7). The bootstrap checks and says so.

Key settings (config\relay.psd1)

Setting	Value	Setting	Value
PollSeconds	30	MaxOutputBytes	5 MB
OutboxCheckSeconds	2	MaxFileBytes	25 MB
StableSeconds	3	CompactThresholdMB	50
DefaultTimeoutSeconds	600	CompactKeepDays	7
MaxTimeoutSeconds	7200	StaleJobMinutes	60